TFE4171 - Design of Digital Systems 2
Project Part A

February 4, 2026

Deadline : Friday, 13th of March, 14:00

Submission

The project is divided into Part A and Part B. This will submission will encapsulate Part A. You will receive feedback and not a final grade from this delivery and you will improve this in part B for the final project grade. Submit Part A report and code on blackboard, Part B report on Inspera and code on Blackboard.

Upload on blackboard, upload your report as a PDF and a .ZIP file with your code. All code must be written by the group and code should be ready to run without errors. You should include the code you used to make your solutions in the PDF, clearly marked for each question along with the explanation of your solutions.

Only one group member needs to submit on blackboard, and you can and should download your submission to check it has been uploaded correctly. The course has firm deadlines and all group work must be your own as you may be asked questions about your submissions in the lab sessions, issues like plagiarism will be reported to the course co-ordinator.

Project Description

You, verification engineers for SuperSoC AG, have been tasked with the verification of an untested HDLC (High-level Data Link Control) module to connect two devices together as part of a large project. You are provided with an HDLC implementation and in order to meet a hastily imposed implementation deadline thanks to your well-intentioned colleagues in the marketing department. It is important to retrofit the code with SystemVerilog Assertions to ensure that the design meets the specification before it is integrated. You are given a lot of flexibility in what assertions you must write, but you must justify each assertion.

Supplementary Materials

Included in Project A and B are documents detailing the specification of the HDLC controller. These documents have been provided on Blackboard:

- `Design description HDLC Controller.pdf`
- `Additional materials for Part A.pdf`

It is likely that you will need to review these documents multiple times while making your assertions. Be sure to go through them carefully to better understand your project tasks.

Getting started

Part A gives an introduction to the term project by requiring concurrent and immediate assertions for the Rx module of the HDLC design.

1. Firstly, in your home directory, copy the project material from the master user:

```
cp -r ../dd2master/termproject/hdlc ~/
```

In hdlc/, you will find two directories. It is important to familiarize yourself with the file names in both directories:

- **rtl**: This contains various RTL circuits for the receive and transmit designs of HDLC.
- **tb**: This includes the file `testPr_hdlc.sv`, where you will write the immediate assertions as instructed later. Additionally, you will find the file `assertions_hdlc.sv`, where you will write your concurrent assertions. This directory is of particular focus, as it contains these two files for writing your assertions.

Read over these files, and in particular focus on figures 1 and 2, for where you should place your code.

```
/*****************************************************************************
 *                                                                           *
 *                              Student code                                 *
 *                                                                           *
 *****************************************************************************/

// VerifyAbortReceive should verify correct value in the Rx status/control
// register, and that the Rx data buffer is zero after abort.
task VerifyAbortReceive(logic [127:0][7:0] data, int Size);
  logic [7:0] ReadData;

  // INSERT CODE HERE

endtask

// VerifyNormalReceive should verify correct value in the Rx status/control
// register, and that the Rx data buffer contains correct data.
task VerifyNormalReceive(logic [127:0][7:0] data, int Size);
  logic [7:0] ReadData;
  wait(uin_hdlc.Rx_Ready);

  // INSERT CODE HERE

endtask

// VerifyNormalReceive should verify correct value in the Rx status/control
// register, and that the Rx data buffer contains correct data.
task VerifyOverflowReceive(logic [127:0][7:0] data, int Size);
  logic [7:0] ReadData;
  wait(uin_hdlc.Rx_Ready);

  // INSERT CODE HERE

endtask
```

Figure 1: `testPr_hdlc.sv`, where you will write your immediate assertions.

```
  /*****************************************
   *  Verify correct Rx_FlagDetect behavior  *
   *****************************************/

  sequence Rx_flag;
    // INSERT CODE HERE
  endsequence

  // Check if flag sequence is detected
  property RX_FlagDetect;
    @(posedge Clk) Rx_flag |-> ##2 Rx_FlagDetect;
  endproperty

  RX_FlagDetect_Assert : assert property (RX_FlagDetect) begin
    $display("PASS: Flag detect");
  end else begin
    $error("Flag sequence did not generate FlagDetect");
    ErrCntAssertions++;
  end

  /*****************************************
   *  Verify correct Rx_AbortSignal behavior  *
   *****************************************/

  //If abort is detected during valid frame. then abort signal should go high
  property RX_AbortSignal;
    // INSERT CODE HERE
  endproperty

  RX_AbortSignal_Assert : assert property (RX_AbortSignal) begin
    $display("PASS: Abort signal");
  end else begin
    $error("AbortSignal did not go high after AbortDetect during validframe");
    ErrCntAssertions++;
  end

endmodule
```

Figure 2: `assertions_hdlc.sv`, where you will write your concurrent assertions.

Project Part A Tasks

Immediate Assertions

For the immediate assertions, you will write assertions to verify the correct values in the Rx status/control register and Rx data buffer, both of which are read using `AddressIF`. The registers are documented in the HDLC design description.

1. Check for correct values after normal operation in the `VerifyNormalReceive` task.
2. Check for correct values after an abort in the `VerifyAbortReceive` task.
3. Check for correct values after an overflow in the `VerifyOverflowReceive` task.

`IMPORTANT:` After implementing the three tasks, proceed with coding the concurrent assertions described in the next section. If you attempt to run the command `./simulate.sh` at this point, you may encounter errors from `assertions_hdlc.sv`, as this file must first be completed with the concurrent assertions from the next section. Do not run this command yet.

Questions Immediate Assertions 1. to 3.

For all implemented immediate tasks **1.** to **3.** show and explain your assertion in your report, including which flags you verified and how you checked the Rx data buffer.

Concurrent Assertions

For the concurrent assertions, you will write assertions to verify the correct behavior of `Rx_FlagDetect` and `Rx_AbortSignal`.

1. Write the `Rx_flag` sequence, which identifies a flag.
2. Write the `Rx_AbortSignal` property, which verifies that the abort signal is raised when an abort pattern is observed during a valid frame.

`IMPORTANT:` After completing both your immediate and concurrent assertions, run `./simulate.sh`. The first time you run the simulation, you may encounter an error about an unknown `RXSC` used in `testPr_hdlc.sv`. This error is intentional. Investigate where this error originates in the code, determine the reason, and proceed to fix it. Document in your report how you resolved the issue.

Questions Concurrent Assertions 1. to 4.

1. and **2.** For all implemented concurrent tasks **1.** and **2.** show and explain your assertions in your report.

3. Explain in your report the difference between immediate and concurrent assertions. Provide at least two key points.

4. Document in your report how you resolved the `RXSC` issue in the simulation.

Result of `./simulate.sh`

When you execute `./simulate.sh`, design files and test files will be compiled, and the simulation will start. For reference, a text file named `"Running compilation and simulation"` is available on Blackboard. Use this file to compare your results and verify that your immediate and concurrent assertions produce the expected output.

References

[1] *QuestaSim User Manual.*

[2] Karianne K. Kragseth. *HDLC Module Design Description.* November 15, 2017. (Available on Blackboard).

[3] NTNU. *Additional materials for Part A.* 2025. (Available on Blackboard).

[4] *HDLC at Wikipedia.* Available at: https://en.wikipedia.org/wiki/HDLC.

[5] *SystemVerilog Language Reference Manual 3.1a.* (Available on Blackboard).

[6] *Modelsim 6.0 Quick Guide* (contains all the commands used when scripting). Available at: https://users.ece.cmu.edu/kbiswas/qk_guide.pdf.

[7] *CRC Calculation at Wikipedia.* Available at: https://en.wikipedia.org/wiki/Computation_of_cyclic_redundancy_checks, https://en.wikipedia.org/wiki/Cyclic_redundancy_check.